ENGINEERING ASSIGNMENT

Real-Time Multilingual Voice AI Agent

Clinical Appointment Booking

Preferred stack: Python · TypeScript

Overview

You are building a real-time voice AI agent for a digital healthcare platform. The agent's primary function is to book and manage clinical appointments through natural voice conversations — entirely without human intervention.

The agent operates across three Indian languages (English, Hindi, Tamil), maintains awareness of patient context both within a session and across past interactions, and can proactively reach out to patients as part of outbound reminder and follow-up campaigns. It must handle real-world messiness: conflicting slots, mid-conversation changes of mind, unclear requests, and graceful recovery from errors.

This is a systems problem as much as it is an AI problem. We care about how you architect the pipeline for low latency, how you design memory so context is genuinely useful, and how cleanly your components are separated. A working demo with shaky internals is not what we are looking for.

> Target end-to-end response latency: under 450 ms from speech end to first audio response. This must be measured, logged, and discussed in your submission.

What to Build

Voice Conversation Agent

A real-time conversational agent that accepts voice input, reasons over it, and responds in voice — capable of managing the full appointment lifecycle: booking, rescheduling, cancellation, and conflict resolution across doctors and time slots.

Multilingual Support

The agent must detect and sustain conversations in English, Hindi, and Tamil. Language preference should carry over across sessions for returning patients.

Contextual Memory

The agent must maintain context at two levels: within an active session (current intent, pending confirmations, conversation state) and across sessions (patient history, preferences, prior interactions). Your design choices here — storage, retrieval, prompt integration — should be clearly documented.

Outbound Campaign Mode

Beyond inbound calls, the agent must be capable of initiating outbound calls for campaigns such as appointment reminders or follow-up check-ins. It should handle patient responses naturally — booking, rescheduling, or logging a polite rejection — and adapt its language dynamically.

Scheduling & Conflict Logic

The agent must reason over availability and prevent invalid bookings. Double-booking, past-time selection, and unavailable doctors are scenarios that must be handled gracefully with alternatives offered where possible.

Deliverables

1. GitHub repository — clean, runnable, with clear project structure
2. Loom walkthrough — up to 3 minutes covering a live demo and a brief architecture overview
3. Architecture diagram — PNG or PDF
4. README — covering setup, architectural decisions, memory design, latency breakdown, tradeoffs, and known limitations

---

Evaluation Criteria

Area	Weight
Real-time voice architecture & latency	**20%**
Agentic reasoning & tool orchestration	**20%**
Memory design	**15%**
Appointment & conflict management	**10%**
Multilingual handling	**10%**
Performance optimisation	**10%**
Code quality & structure	**10%**
Documentation & README	**5%**

Bonus

- Interrupt / barge-in handling mid-response
- Redis-backed memory with TTL
- Horizontal scalability design or cloud deployment
- Background job queues for campaign scheduling

Constraints

- Tool-calling and agent orchestration must be genuinely implemented — not simulated with hardcoded responses
- Reasoning traces must be visible and demonstrable

> We are evaluating systems thinking, latency awareness, and architectural clarity — not just whether the demo works.